

- **main** — stable/demo-ready; no direct work.
- **develop** — integration branch for tested feature merges.
- Four feature branches remain isolated until each slice is ready.
- If a dependency is unfinished, use the mock adapter and continue.

Git workflow

```
main ↑ develop ↑ ↑ ↑ ↑ backend customer ai operations branch branch branch branch Contract → Mock → Build → PR → Test
→ Merge
```

Git rules

- No force-push to main/develop.
- Small commits using feat:, fix:, refactor:, docs:.
- Never commit .env or secrets.
- Every merged branch must build successfully.
- Keep PRs small and reversible.

5. Detailed Team Responsibilities

Person 1 — Backend + Supabase

- Create schema, relationships, indexes, seed data, Auth and Storage setup.
- Implement Spring Boot controllers/services/repositories, validation and role authorization.
- Own appointment rules, queue state machine and server-side wait calculations.
- Expose APIs for customer, salon and admin modules.
- Implement Realtime event strategy and notification records.
- Publish API examples and mock datasets so other developers can integrate independently.

Person 2 — Customer Full Flow

- Build Login → Home → Services → AI → Recommendations → Booking → Token → Live Queue → Notifications → Feedback.
- Create typed customer service layer with mock and real implementations.
- Make UI mobile-first and QR-friendly.
- Own loading, empty, error and success states.
- Swap mock calls for real APIs without rewriting screens.

Person 3 — AI + Intelligence

- Integrate vision/LLM API with structured JSON output.
- Analyze face shape, hair type and density for recommendations.
- Match results only to catalog hairstyle IDs.
- Implement optional virtual preview if reliable.
- Implement wait-time intelligence and congestion rules.
- Generate owner-facing insights from structured metrics; do not claim custom ML training.

Person 4 — Admin + Salon Panel

- Build live salon queue and Call Next / Start / Complete actions.
- Show customer service + selected hairstyle + AI recommendation.
- Build admin KPIs, appointments, revenue, wait, utilization and popular services.
- Start with mocks, then connect to APIs.
- Own tablet/desktop usability and demo controls.

6. Database Design

Table	Important fields / purpose
users	id UUID PK; name; email; phone; role; profile_image; created_at. Roles CUSTOMER, STAFF, ADMIN.
salons	id UUID PK; name; address; phone; opening_time; closing_time; status; created_at.
staff	id UUID PK; user_id FK; salon_id FK; specialization; status; created_at. AVAILABLE/BUSY/BREAK/OFFLINE.
services	id UUID PK; salon_id FK; name; description; price; duration_minutes; category; image_url; is_active.
hairstyles	id UUID PK; name; description; image_url; category; suitable_face_shapes; suitable_hair_types.
appointments	id UUID PK; customer_id; salon_id; service_id; staff_id; appointment_date; appointment_time; status; source; created_at.
queue_tokens	id UUID PK; salon_id; customer_id; appointment_id; token_number; service_id; staff_id; queue_date; status; position; estimated_wait; timestamp.
ai_recommendations	id UUID PK; customer_id; hairstyle_id; face_shape; hair_type; hair_density; match_score; reason; image_url; created_at.
notifications	id UUID PK; user_id; title; message; type; is_read; created_at.
feedback	id UUID PK; customer_id; appointment_id; rating; waiting_rating; service_rating; comment; created_at.
queue_events	id UUID PK; token_id; event_type; timestamp. JOINED/CALLED/STARTED/COMPLETED/CANCELLED.

Relationships

- Salon → many staff/services/appointments/tokens.
- Customer → many appointments/tokens/AI recommendations/notifications.
- Appointment → optional queue token at check-in.
- Queue token → service + optional staff.
- AI recommendation → catalog hairstyle → real salon service.

7. REST API Contract

Domain	Method + endpoint	Purpose
Auth	POST /api/auth/register	Register
Auth	POST /api/auth/login	Login
Services	GET /api/services	List services
Services	POST/PUT/DELETE /api/services/{id}	Manage services
Appointments	POST /api/appointments	Create booking
Appointments	GET /api/appointments/customer/{id}	Customer bookings
Appointments	GET /api/appointments/salon/{id}	Salon bookings
Appointments	PUT /api/appointments/{id}/status	Booking status
Queue	POST /api/queue/join	Create token
Queue	GET /api/queue/live/{salonId}	Live queue
Queue	GET /api/queue/token/{tokenId}	Token details
Queue	PUT /api/queue/{tokenId}/call	Call customer
Queue	PUT /api/queue/{tokenId}/start	Start service
Queue	PUT /api/queue/{tokenId}/complete	Complete service
Staff	GET /api/staff	List staff
Staff	PUT /api/staff/{id}/status	Staff status
AI	POST /api/ai/analyze-hairstyle	Analyze selfie/recommend
AI	POST /api/ai/preview-hairstyle	Optional preview
Analytics	GET /api/analytics/dashboard	Owner KPIs
Analytics	GET /api/analytics/queue	Queue metrics
Analytics	GET /api/analytics/services	Service metrics

8. Shared JSON Contracts

```
Queue join: { "tokenId":"uuid", "tokenNumber":108, "position":4, "estimatedWait":24, "status":"WAITING" }

AI recommendation: { "faceShape":"Oval", "hairType":"Wavy", "hairDensity":"Medium",
"recommendations":[{"hairstyleId":"HS01","name":"Textured Crop", "matchScore":95,"reason":"Works well with oval face
shape and wavy hair"}] }
```

Contract rule: field names/types are frozen early. Frontend mocks use the exact same shape as the real API.

9. Core Functional Flows

Customer AI → Booking

Upload selfie → AI analysis → catalog matching → 3–5 recommendations → select style → map to actual service → appointment or queue → token/booking → live tracking.

Online Booking

Select salon → service → date/time → availability validation → appointment BOOKED/CONFIRMED → check-in → queue token when required.

Walk-In

Scan salon QR → select service → optional AI → request token → backend assigns number → WAITING → live queue → notifications.

Salon

Live queue → Call Next → CALLED → Start Service → IN_SERVICE → Complete → COMPLETED → queue recalculation.

Admin

Dashboard → customers/appointments/revenue/wait → utilization/service popularity → congestion/AI insight → staffing/service action.

10. Queue State Machine

WAITING → CALLED → IN_SERVICE → COMPLETED ■ ■ ■■■■■■■■■■ → CANCELLED ■■■■■■■■■■ → NO_SHOW

Spring Boot authoritatively performs transitions. Each important transition creates a queue_events record. Realtime pushes the new state to clients.

11. Intelligence Layer

Wait-time MVP: sum expected durations of customers ahead ÷ available staff capacity. Example: 20 + 30 + 15 minutes with two stylists = 32.5 minutes.

- Use historical average duration per service/stylist when enough data exists.
- Congestion inputs: current queue, upcoming appointments, historical demand, available staff.
- Rule-based congestion is acceptable for the MVP; label it honestly as an operations intelligence engine.
- Example insight: “Peak demand expected around 6:30 PM. Recommendation: move one available stylist to haircut service.”

12. Notifications

Mandatory: in-app notifications stored in Supabase. Booking confirmed; position changed; “2 customers away”; “It’s your turn.” Browser Web Push is bonus. SMS/WhatsApp are not core dependencies.

13. Screen Map

Customer	Salon	Admin
Login / Home	Login / Live Queue	Login / Dashboard
Services	Token Details	Appointments
AI / Recommendations	Call / Start / Complete	Live Operations
Booking / Token	Staff Status	Staff Management
Live Queue / Notifications	Queue History	Service Management
Feedback / Profile	Customer Details	Analytics / AI Insights

14. 24-Hour Parallel Execution Plan

Time	Person 1	Person 2	Person 3	Person 4
0–2h	DB + contract + backend skeleton	Customer components + mocks	AI proof + schema	Dashboard shell + mocks
2–6h	Auth/services/appointments	Login/services/booking	AI analysis + catalog mapping	Salon queue + admin shell
6–10h	Queue APIs + seed data	Booking/token integration	Recommendations + intelligence	Queue actions + API integration
10–14h	Realtime + queue rules	Live queue + notifications	Wait + congestion	Staff status + operations
14–18h	Analytics APIs + fixes	Feedback + polish	AI insights + optional preview	Admin analytics
18–21h	Integration/debug	E2E customer test	AI fallback/test	E2E operations test
21–24h	Deployment support	Demo + UI polish	Demo AI + backup	Dashboard demo + presentation

15. Integration Gates

- Hour 0–1: freeze API/DB contracts.
- Hour 6–8: login → services → booking works.
- Hour 10–12: booking/check-in → token → live queue.
- Hour 12–15: selfie → AI → hairstyle → service mapping.
- Hour 15–18: salon operations → customer updates → analytics.
- Hour 18–21: full end-to-end/realtime testing.
- Hour 21–24: no major features; deploy, polish, rehearse, fix.

16. Mock Adapter Strategy

Customer UI → customerService → MockCustomerService (early) ■ ApiCustomerService → Spring Boot (integration) AI UI → aiService → MockAIService (early) ■ ApiAIService → Spring Boot → AI API

The interface stays identical, so Person 2/4 never wait for Person 1 and Person 2 never waits for Person 3.

17. Security & Reliability

- Role authorization in Spring Boot, not only hidden frontend routes.
- Validate customer ownership and salon/staff/service IDs.
- Supabase Auth; never store raw passwords.
- AI/API credentials only in environment variables.
- Never commit .env or service-role secrets.
- Use controlled Storage paths for images.
- Prepare seeded data and mock-AI fallback for external API failure.

18. Testing Checklist

Area	Acceptance tests
Auth	Valid/invalid login; role restrictions; session failure
Booking	Valid/unavailable slot; invalid service; cancellation
Queue	Unique token; position; transitions; cancel/no-show
Realtime	Queue update visible without refresh
AI	Valid image; timeout/malformed response; catalog-only recommendations
Notifications	Booking; position; turn approaching; turn called
Dashboard	KPIs; empty states; filters; authorization
Responsive	Customer phone; salon tablet; admin laptop/desktop

19. Deployment

- Next.js frontend to web hosting; Spring Boot to Java-compatible hosting.
- Supabase for PostgreSQL/Auth/Storage/Realtime.
- AI provider key supplied through deployment environment.
- Seed salon, stylists, services, hairstyles, customers and queue tokens before judging.
- Keep a backup demo path using seeded/mock data.

20. Winning Demo Script

Step	Demo action
1	Customer: "I don't know what haircut suits me."
2	Upload selfie.
3	Show 3 recommendations with scores/reasons.
4	Select Textured Crop → Book This Style.
5	Show token #108 + estimated wait.
6	Simulate customers; live queue changes.
7	Show congestion prediction + staffing recommendation.
8	Customer gets "2 customers away" notification.
9	Stylist sees customer + service + selected hairstyle.
10	Complete → feedback → owner dashboard.

21. Definition of Done

- Customer completes login → AI → booking/token → queue → feedback.
- AI recommendations map to real hairstyle/service IDs.
- Online and walk-in customers coexist in one queue model.
- Salon can call/start/complete and customer receives updates.
- Estimated wait recalculates after queue changes.
- In-app notifications work independently of browser push.
- Admin shows meaningful KPIs and one actionable insight.
- All four feature branches build successfully and are merged into develop.
- main remains stable/demo-ready and contains no secrets.

22. Final Engineering Principle

Build independently, integrate continuously. The shared API/JSON contract—not another developer's unfinished code—is the dependency boundary. Person 1 owns the backend implementation; Person 2 and Person 4 build with mocks; Person 3 builds against the AI schema and catalog. This keeps all four developers productive for the full 24 hours.

Final stack: Next.js responsive web/PWA • Java Spring Boot • Supabase PostgreSQL/Auth/Storage/Realtime • External AI API • Git with four isolated feature branches.